

02 — Perancangan Basis Data

Tujuan: Menjelaskan **keputusan desain** yang diambil saat merancang skema, sehingga database menjadi **konsisten, efisien, dan mudah dikembangkan**.

1. Prinsip Perancangan yang Diikuti

Prinsip	Penerapan di BelajarKUY
Single Source of Truth	Profil user disimpan di 1 tabel users saja (bukan dipisah per role)
Atomicity	Setiap kolom menyimpan 1 nilai → tidak ada array/CSV → memenuhi 1NF
No Redundancy	Harga kursus tidak digandakan di tiap order; hanya snapshot harga (original_price , final_price) disimpan saat transaksi
Referential Integrity	Semua relasi pakai FOREIGN KEY dengan ON DELETE policy yang jelas
Naming Convention	Snake_case, jamak untuk tabel (users , courses), _id suffix untuk FK
Indexed Lookup	Kolom yang sering difilter punya index (status, role, slug, email)

2. Strategi Desain per Domain

🔑 A. Manajemen User dengan ROLE (bukan tabel terpisah)

Keputusan: Satu tabel **users** dengan kolom **role** `ENUM('user', 'instructor', 'admin')`.

Alasan:

- Hindari duplikasi (user bisa naik level jadi instruktur tanpa migrasi data).
- Query JOIN lebih sederhana (1 tabel **users** saja).
- Index pada **role** mempercepat filter "ambil semua instruktur".

```
users (id, name, email, password, role, photo, phone, address, bio, website, ...)
    PRIMARY KEY (id), UNIQUE (email), INDEX (role)
```

📁 B. Struktur Kursus Berhierarki

```
courses (1) → (N) course_sections (1) → (N) course_lectures
    ↘
```

(1) → (N) course_goals

Keputusan: Pisah `course_sections` & `course_lectures`, JANGAN simpan semua materi di kolom JSON.

Alasan:

- Mudah query "berapa lecture di kursus X?"
- Mudah tracking penyelesaian per lecture (`lecture_completions`).
- Reorder lecture cukup ubah kolom `sort_order`.

C. Pemisahan PAYMENTS dan ORDERS (kritikal!)

Keputusan: Buat 2 tabel terpisah:

- `payments` = 1 transaksi Midtrans (1 invoice gabungan).
- `orders` = item per kursus dalam 1 payment.

```
payments (id, user_id, midtrans_order_id, total_amount, status, ...)
          ↓ 1:N
orders (id, payment_id, course_id, instructor_id, coupon_id, final_price, ...)
```

Alasan:

- 1 user bisa checkout 3 kursus sekaligus → 1 payment Midtrans, 3 baris order.
- Bagi-hasil instruktur dihitung per `order.instructor_id`.
- Refund parsial → cukup ubah status di salah satu order saja.

D. Enrollment Sebagai Tabel Tersendiri

Keputusan: Tidak simpan "user telah beli kursus" di tabel `orders`. Dibuat tabel `enrollments` terpisah.

Alasan:

- `orders` = catatan transaksi (immutable, audit).
- `enrollments` = HAK AKSES belajar (dapat diberi gratis tanpa order, misalnya promosi).
- Query "kursus apa saja yang bisa diakses user X?" cukup `WHERE user_id = X` di `enrollments`.

★ E. UNIQUE Constraint untuk Duplikasi

Tabel	UNIQUE	Alasan
<code>wishlists (user_id, course_id)</code>	1 kursus tidak boleh masuk wishlist 2×	
<code>carts (user_id, course_id)</code>	1 kursus tidak boleh masuk cart 2×	
<code>enrollments (user_id, course_id)</code>	1 user tidak boleh enroll kursus yang sama 2×	

Tabel	UNIQUE	Alasan
<code>reviews (user_id, course_id)</code>	1 user hanya boleh review 1× per kursus	
<code>lecture_completions (user_id, lecture_id)</code>	Cegah double-count progres	

F. Soft-Snapshot Harga di ORDERS

Keputusan: Simpan `original_price`, `discount_amount`, `final_price` di tabel `orders` saat transaksi terjadi.

Alasan:

- Jika instruktur ubah harga kursus besok, harga lama tetap tercatat di order.
- Penting untuk laporan keuangan & dispute.

G. Image di Cloudinary, bukan di Server

Keputusan: Setiap entitas yang punya gambar menyimpan:

- `image_url` (URL CDN Cloudinary)
- `image_public_id` (ID untuk delete/update)

Alasan:

- Server Laravel tidak terbebani file storage.
- CDN otomatis = load cepat di seluruh dunia.
- `public_id` memungkinkan hapus file lama saat update.

H. CMS via Tabel Konten Terpisah

`sliders`, `info_boxes`, `partners`, `site_infos` = data tampilan yang sering berubah, tidak terikat business logic.

Alasan dipisah: Admin bisa update tampilan homepage tanpa menyentuh tabel transaksional.

3. Konvensi Penamaan

Aturan	Contoh
Tabel pakai plural snake_case	<code>users</code> , <code>course_sections</code>
Primary key bernama <code>id</code> (<code>BIGINT UNSIGNED AUTO_INCREMENT</code>)	<code>id</code>
Foreign key = <code>{tabel_singular}_id</code>	<code>user_id</code> , <code>course_id</code>
Boolean berakhiran tanpa <code>is_</code>	<code>status</code> , <code>bestseller</code> , <code>featured</code>
Timestamp standar Laravel	<code>created_at</code> , <code>updated_at</code>

Aturan	Contoh
ENUM untuk state machine	<code>status ENUM(...)</code>
Kolom uang	<code>DECIMAL(12,2)</code> — bukan FLOAT!

4. Strategi Indeks (untuk Performa Query)

Tabel	Index	Tujuan
<code>users</code>	<code>role</code>	Filter cepat: "ambil semua instruktur"
<code>users</code>	<code>email UNIQUE</code>	Login & cegah email duplikat
<code>courses</code>	<code>(status, featured)</code> composite	Query homepage "kursus featured aktif"
<code>courses</code>	<code>(status, bestseller)</code> composite	Query "bestseller aktif"
<code>payments</code>	<code>(user_id, status)</code> composite	Riwayat pembayaran user
<code>orders</code>	<code>(user_id, status)</code> composite	Dashboard "kursus yang saya beli"
<code>orders</code>	<code>(instructor_id, status)</code> composite	Dashboard instruktur "penjualan saya"
<code>sessions</code>	<code>user_id, last_activity</code>	Manajemen session Laravel

5. Strategi ON DELETE / ON UPDATE

Relasi	Policy	Alasan
<code>courses.category_id → categories.id</code>	CASCADE	Kategori dihapus → kursus ikut hilang (admin-only action)
<code>courses.subcategory_id → sub_categories.id</code>	SET NULL	Sub-kategori opsional, kursus tetap ada
<code>courses.instructor_id → users.id</code>	CASCADE	Instruktur dihapus → kursusnya juga (data sensitif)
<code>orders.payment_id → payments.id</code>	CASCADE	Payment hilang → order tidak valid
<code>orders.coupon_id → coupons.id</code>	SET NULL	Kupon kadaluarsa boleh dihapus, order tetap simpan riwayat
<code>enrollments.*</code>	CASCADE	Akses ikut hilang kalau course/user dihapus
<code>lecture_completions.*</code>	CASCADE	Progress otomatis hilang kalau lecture/user dihapus

✓ Kesimpulan Perancangan

Database BelajarKUY dirancang dengan pendekatan **3NF + Performance-Aware**:

1. ✓ Tidak ada redundansi data

2. ☒ Setiap relasi punya foreign key & cascade policy yang tepat
3. ☒ Index strategis di kolom-kolom hot query
4. ☒ Snapshot harga & data audit terjaga di tabel transaksional
5. ☒ Skema fleksibel untuk pertumbuhan (multi-role user, multi-item order)

→ Hasilnya: database **aman untuk transaksi finansial, cepat untuk query dashboard**, dan **mudah di-maintain**.